

Deep Learning

Lecture 1

What This Course Offers?

Tentative Course Outline

Week #	Topics to be covered
1	Introduction to Deep Learning
2	Basics of Neural Networks
3-4	Optimization algorithms, Hyperparameter tuning, and practical aspects of Deep learning
5-6	Convolution Neural Networks (CNNs)
7	CNN Architectures and Transfer Learning
8-9	Deep learning for Computer Vision
10-12	Sequence Models
13-14	Generative Deep Learning
15-16	Presentations

Prerequisites

- **The course is self-contained.**
- **No single topic is too hard by itself.**
- **Good background in the following is a plus:**
 - Calculus and linear algebra
 - Algorithms
 - Probability and statistics
- **Programming:**
 - You should be able to write non-trivial programs (in Python)

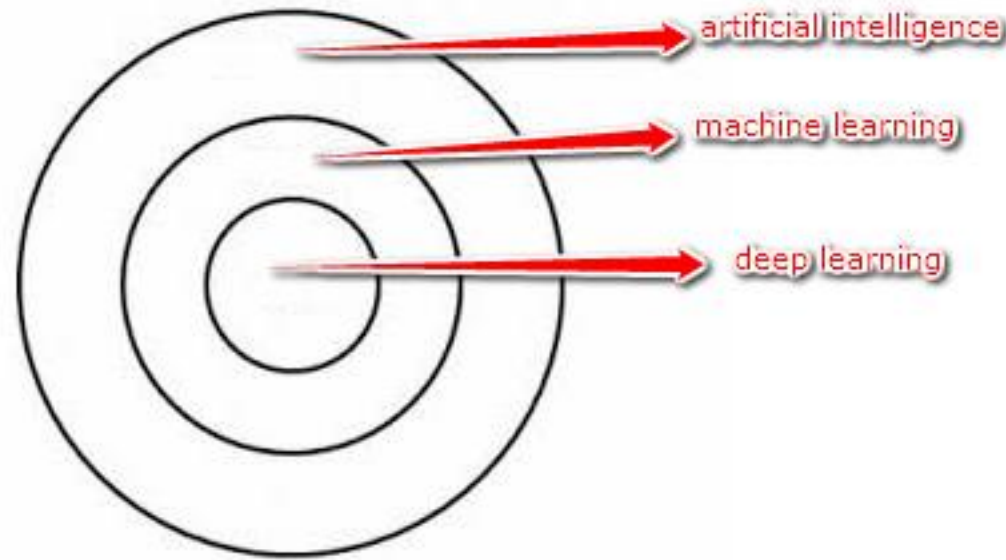
Grading Criteria

• Assignments/ Project	20 - 25%
• Quizzes	10%
• Midterm	25%
• Final	40 - 45%

Course Logistics

- Email: saira.karim@nu.edu.pk
- Course Content: Google classroom (slides, books, additional reading material, practice problems, assignment statements)
 - Class: Deep Learning Fall 2026
- Course evaluations: Flex
- Course attendance: Flex

AI vs Machine Learning vs Deep Learning



- **Artificial Intelligence (AI):** AI is the broad term, applying to any technique that enables computers to mimic human intelligence, using logic, if then rules, and machine learning including deep learning.

AI vs Machine Learning vs Deep Learning

- **Machine Learning:** the subset of AI, that enables machines to learn by themselves using the provided data and make accurate predictions (improve at tasks with experience).
- **Deep Learning:** the subset of machine learning that composed of algorithms that permits software to train itself to perform task like speech, and image recognition by exposing multilayered neural networks to vast amount of data.
 - DL algorithms are roughly inspired by the information processing patterns found in the human brain.
 - Just like we use our brains to identify patterns and classify various types of information, deep learning algorithms can be taught to accomplish the same tasks for machines.

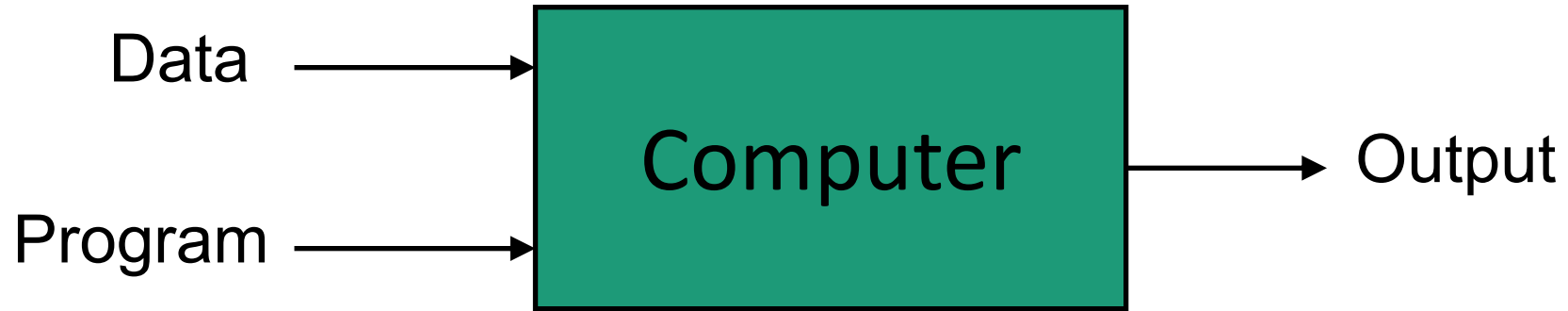
Traditional ML vs Deep Learning

- Traditional ML requires manual feature extraction/engineering
- Feature extraction for unstructured data is very difficult
- Deep learning can automatically learn features in data
- Deep learning is largely a "black box" technique, updating learned weights at each layer

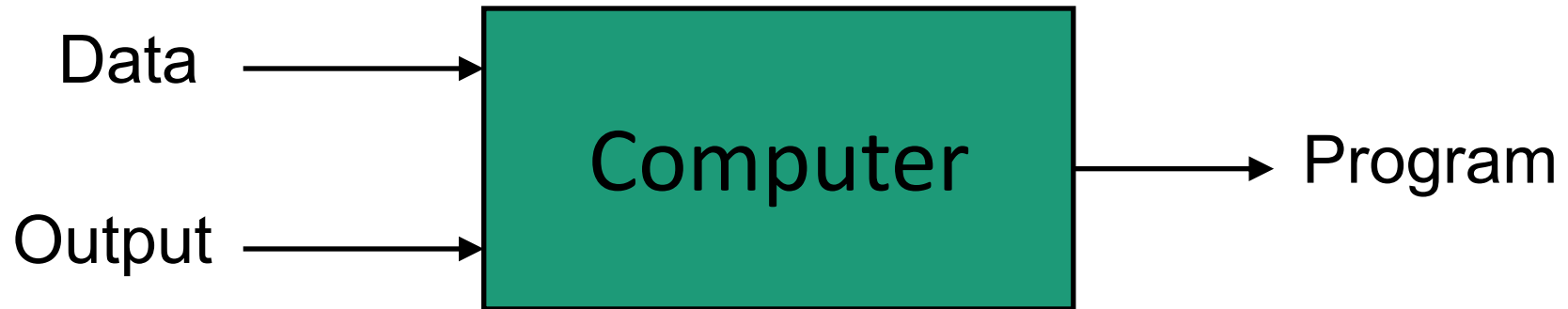
So What Is Machine Learning?

- Automating automation
- Getting computers to program themselves
- Let the **data** do the work!

➤ Traditional Programming



➤ Machine Learning



Practical Applications of Machine Learning

- Spam filtering
- Speech/handwriting recognition
- Object detection/recognition
- Weather prediction
- Stock market analysis
- Search engines (e.g, Google)
- Ad placement on websites
- Credit-card fraud detection
- Webpage clustering (e.g., Google News)
- Social Network Analysis
- Recommendation systems (e.g., Netflix, Amazon)
- Automatic vehicle navigation
- Large Language models
- Generative AI

Machine Learning definition

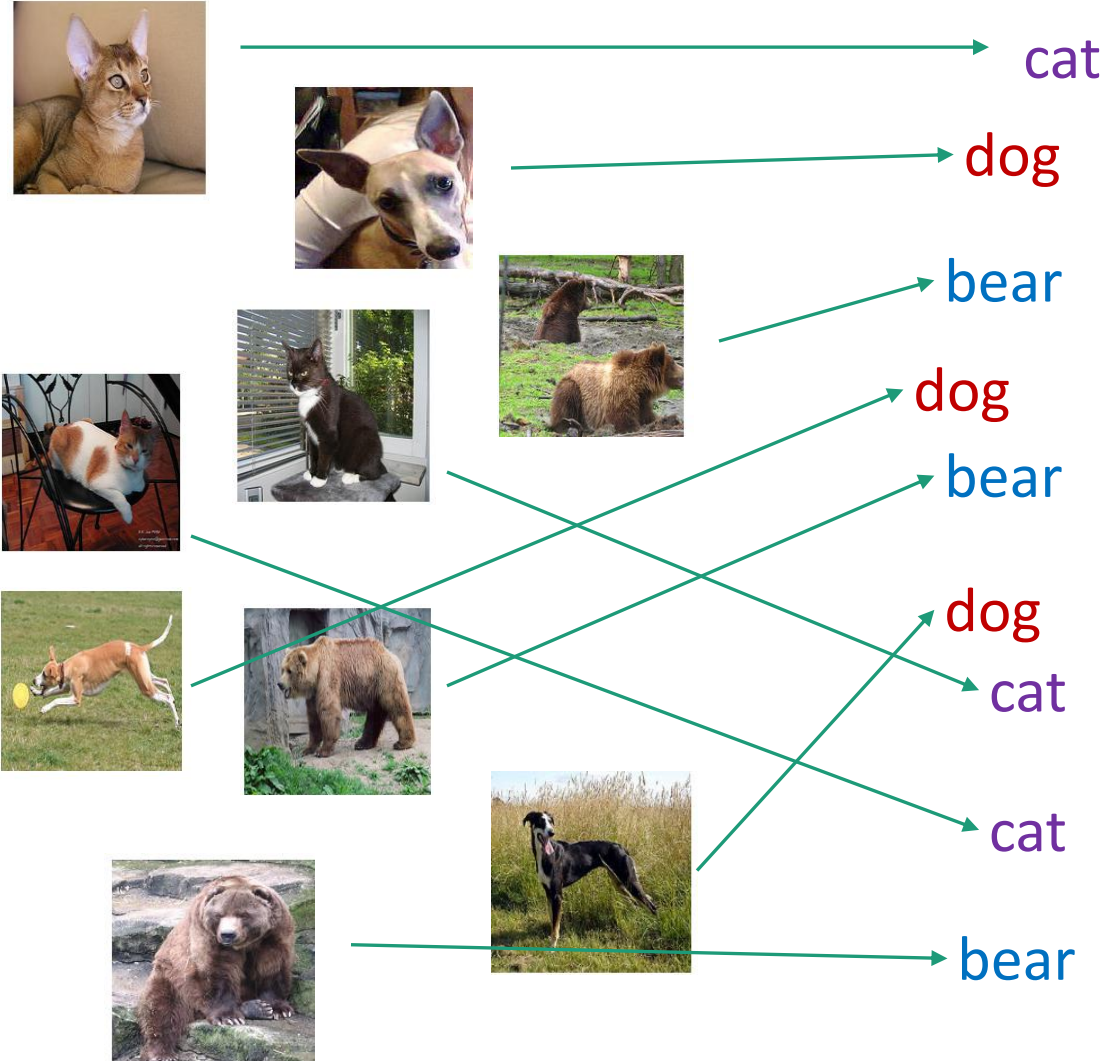
- Tom Mitchell (1998) Well-posed Learning Problem: A computer program is said to *learn* from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

Machine learning algorithms:

- Supervised learning
- Unsupervised learning

Supervised Learning vs Unsupervised Learning

$x \rightarrow y$

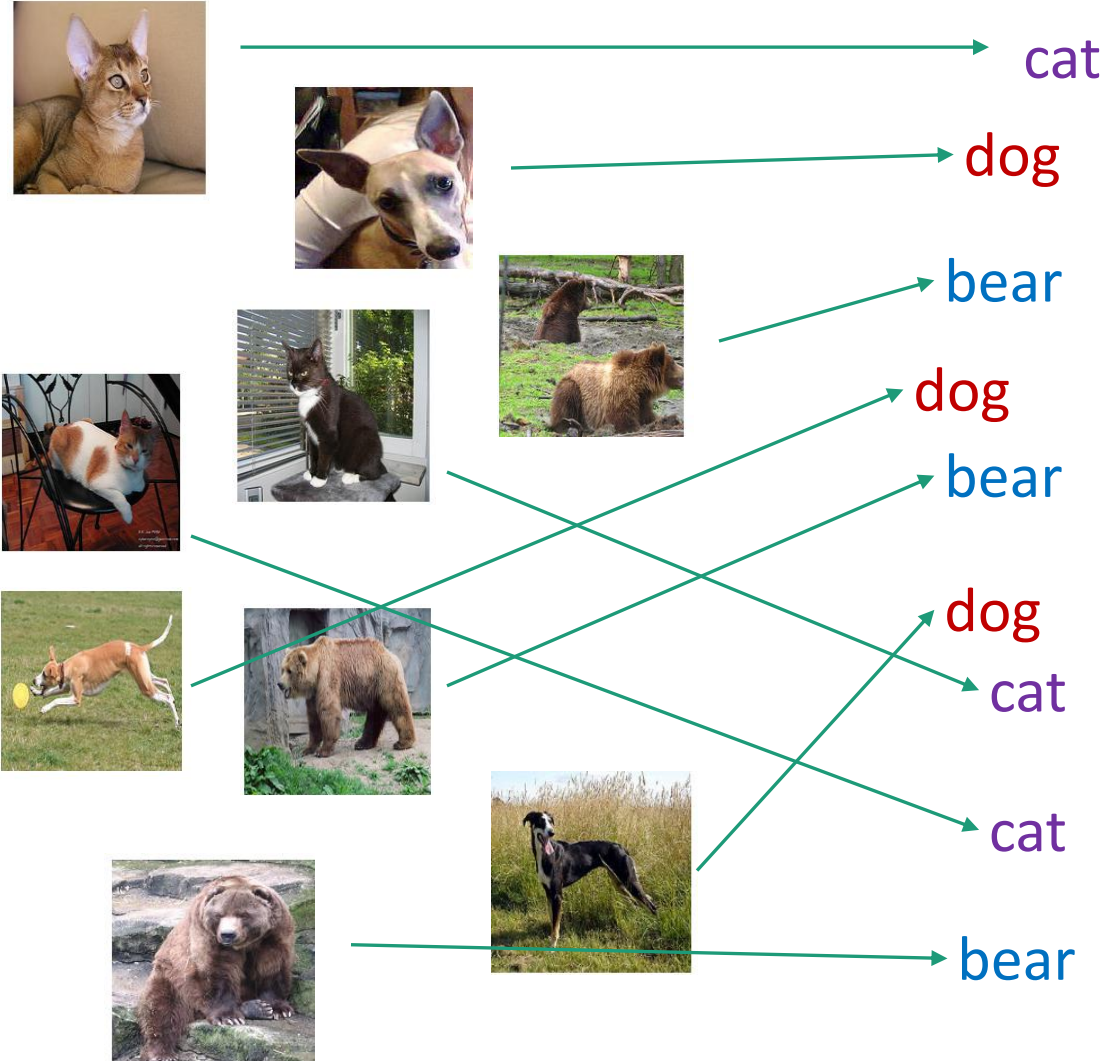


x

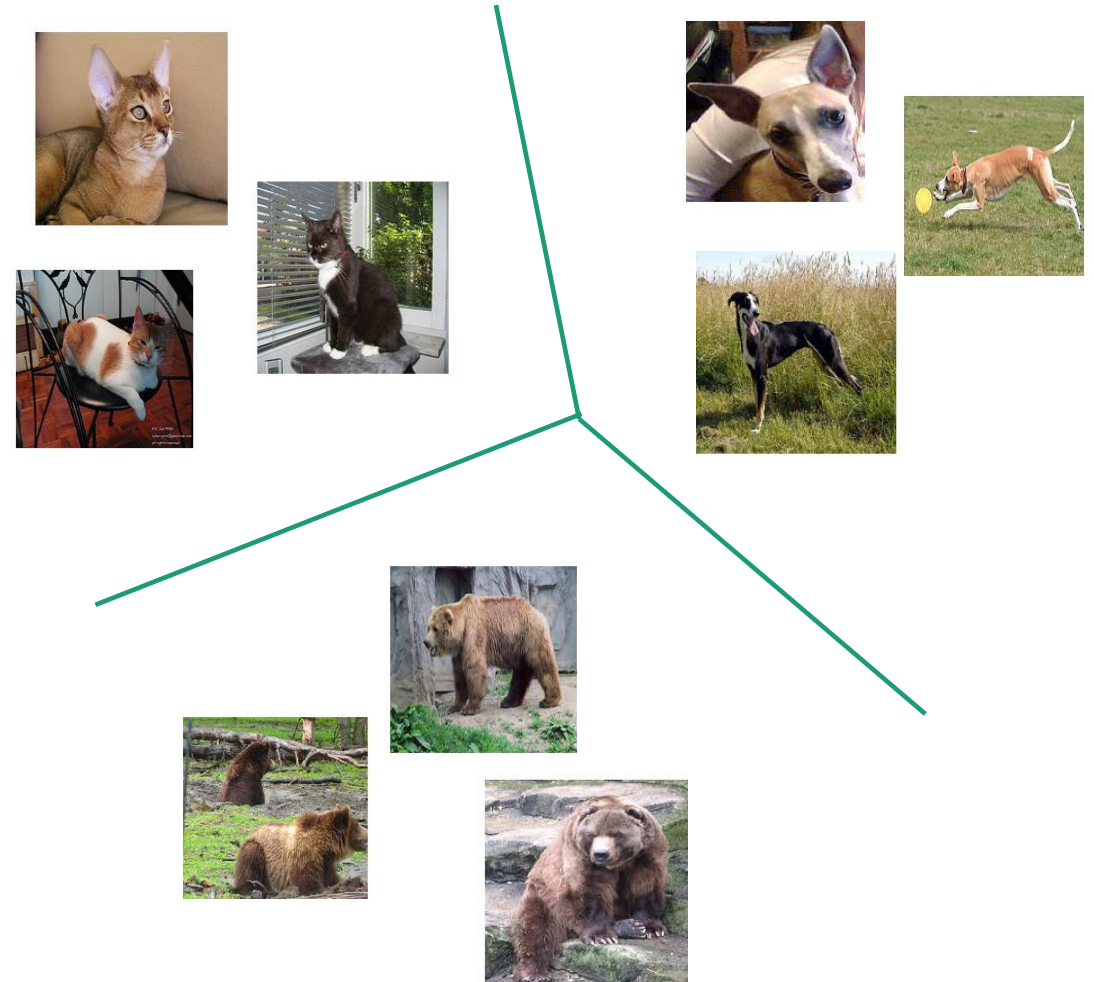


Supervised Learning vs Unsupervised Learning

$x \rightarrow y$

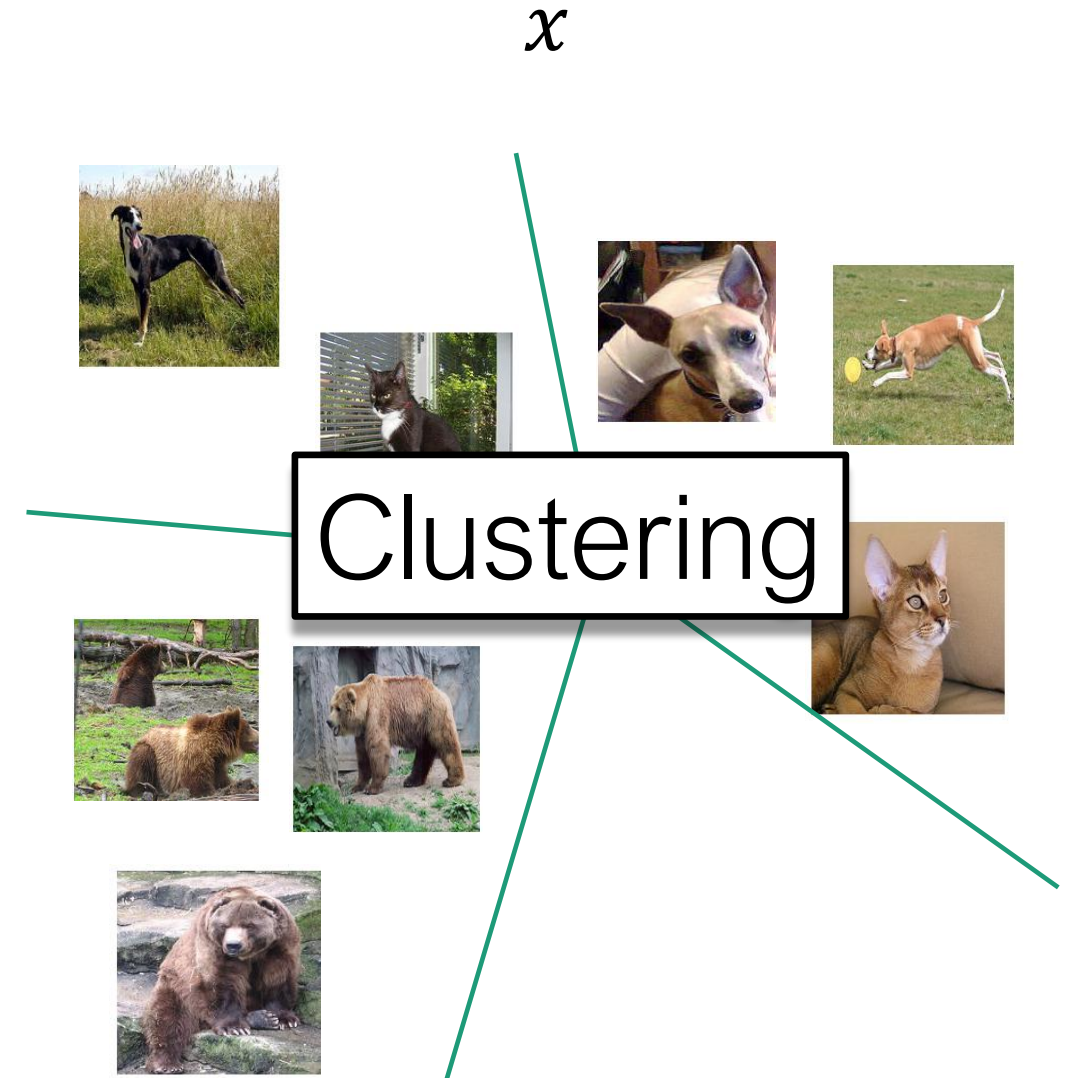
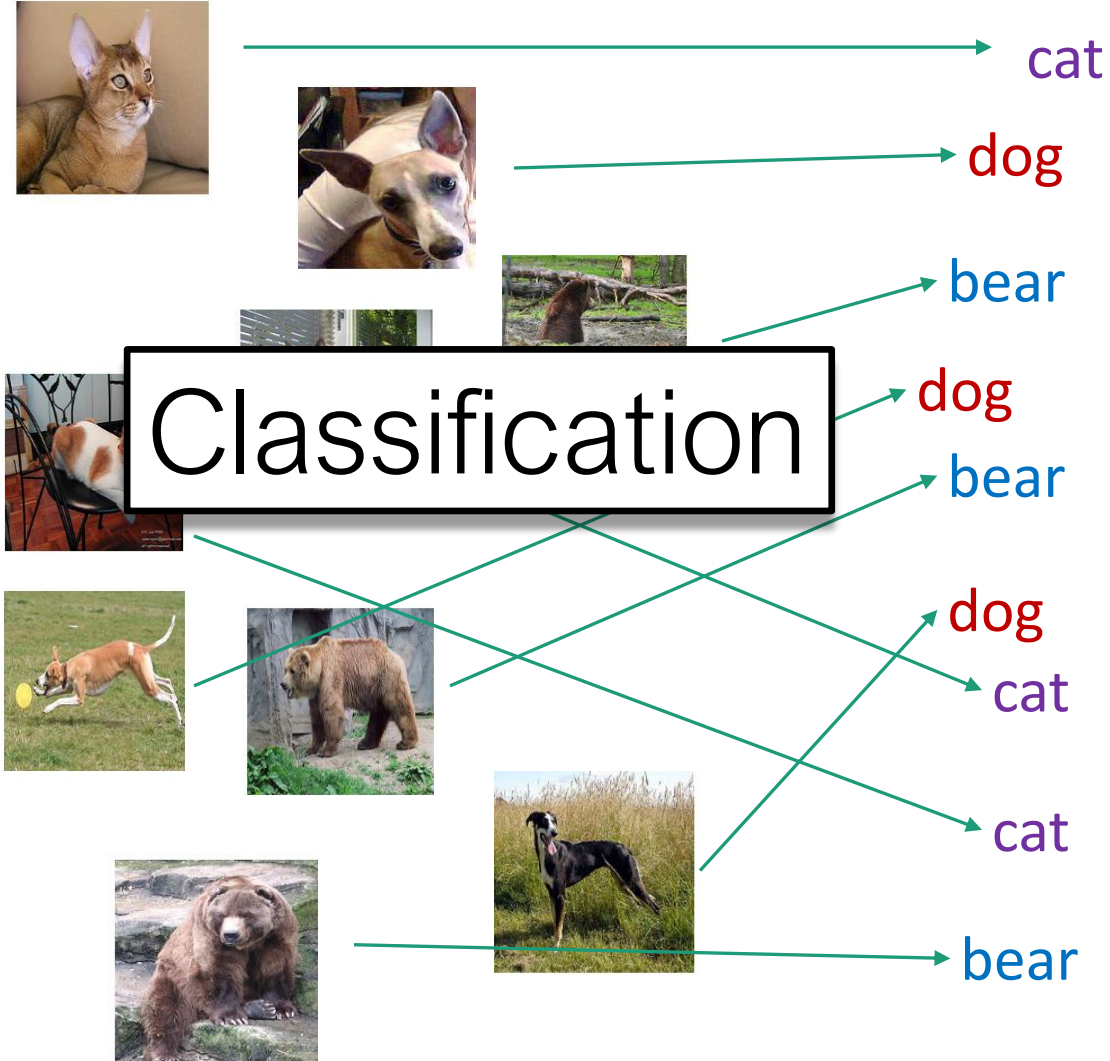


x



Supervised Learning vs Unsupervised Learning

$$x \rightarrow y$$



Supervised Learning vs Unsupervised Learning



Classification

cat

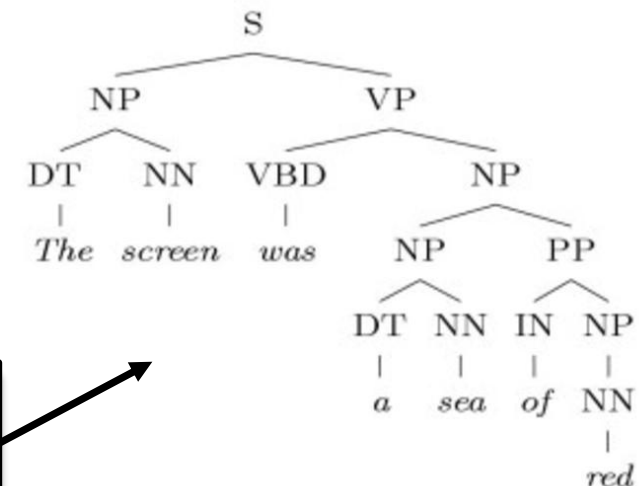


Face Detection



The screen was
a sea of red

Language Parsing



Structured Prediction

Supervised Learning vs Unsupervised Learning

$$\text{cat} = f(\text{img})$$



$$= f(\text{img})$$



Machine Learning as Optimization

- **Supervised learning:** we are given input x , and the goal is to predict label y .
- **Input x can be:**
 - Vectors of real numbers
 - Sequences (natural language)
 - Matrices (images)
 - Graphs (potentially with node and edge features)
- **We formulate the task as an optimization problem.**
 - Find/learn a function that can map input x to labels y
 - Define an objective function (Loss function)

Supervised Learning - Classification

Training Data

cat



dog



bear



cat



cat



dog



bear



dog



bear



Test Data



Supervised Learning - Classification

Training Data



cat



dog



cat

•

•

•



bear

Test Data



•

•

•



Supervised Learning - Classification

Training Data

$$x_1 = [\text{] \quad y_1 = [\text{cat}]$$

$$x_2 = [\text{] \quad y_2 = [\text{dog}]$$

$$x_3 = [\text{] \quad y_3 = [\text{cat}]$$

•
•
•

$$x_n = [\text{] \quad y_n = [\text{bear}]$$

Supervised Learning - Classification

Training Data

inputs	targets / labels / ground truth	predictions
$x_1 = [x_{11} \ x_{12} \ x_{13} \ x_{14}]$	$y_1 = 1$	$\hat{y}_1 = 1$
$x_2 = [x_{21} \ x_{22} \ x_{23} \ x_{24}]$	$y_2 = 2$	$\hat{y}_2 = 2$
$x_3 = [x_{31} \ x_{32} \ x_{33} \ x_{34}]$	$y_3 = 1$	$\hat{y}_3 = 2$
⋮		
$x_n = [x_{n1} \ x_{n2} \ x_{n3} \ x_{n4}]$	$y_n = 3$	$\hat{y}_n = 1$

We need to find a function that maps x and y for any of them.

$$\hat{y}_i = f(x_i; \theta)$$

Learnable
parameters

How do we "learn" the parameters of this function?

We choose ones that makes the following quantity small:

$$\sum_{i=1}^n \mathcal{L}(\hat{y}_i, y_i)$$

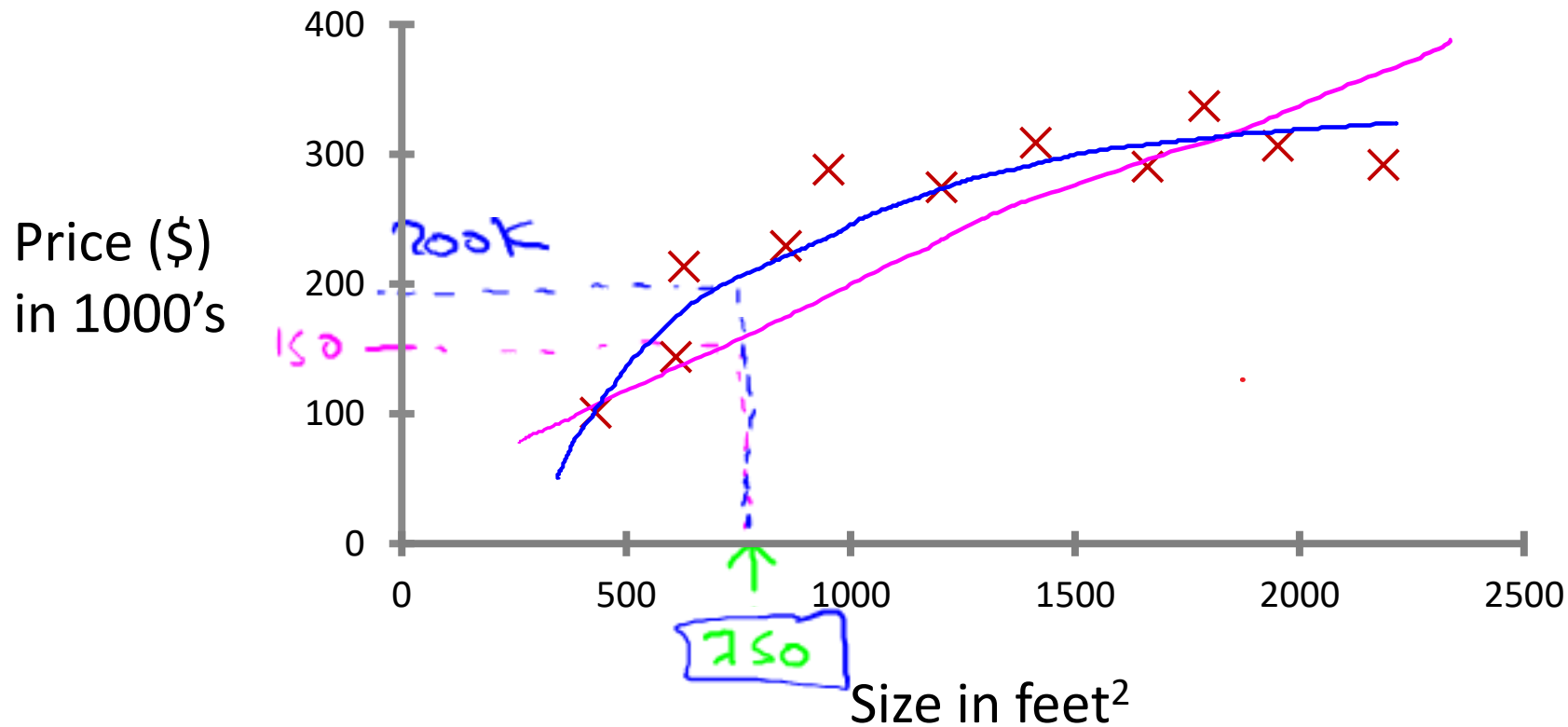
Loss function

Supervised Learning - Classification

- Learning Phase
 - Initialize learnable parameters: θ
 - Compute the predicted labels
 - Compute the loss function
 - Adjust θ to minimize the loss function
- How to optimize loss function: $\min_{\theta} \mathcal{L}(\hat{y}_i, y_i)$
 - Gradient descent

Supervised Learning Example

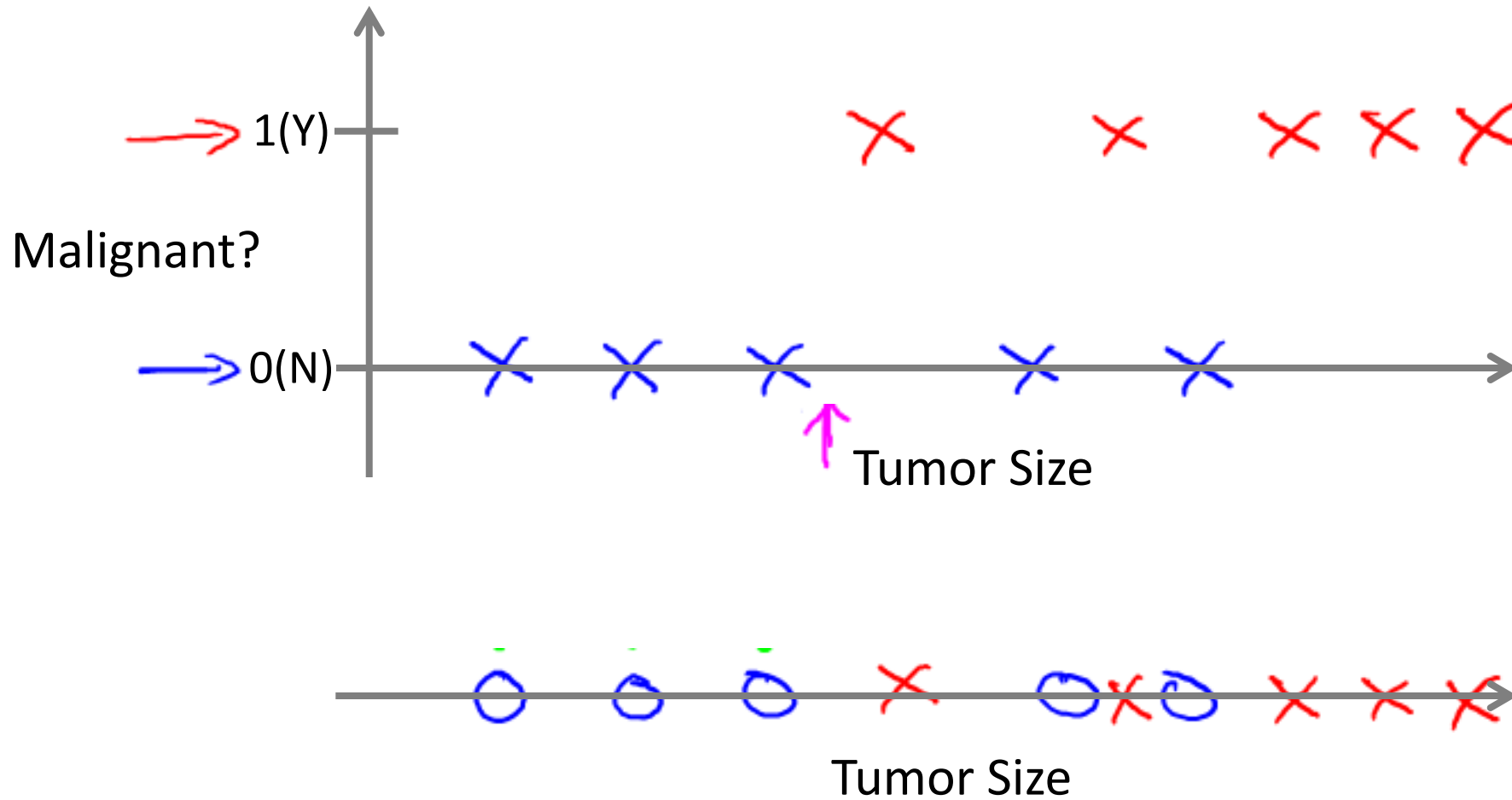
Housing price prediction.



Supervised Learning
“right answers” given

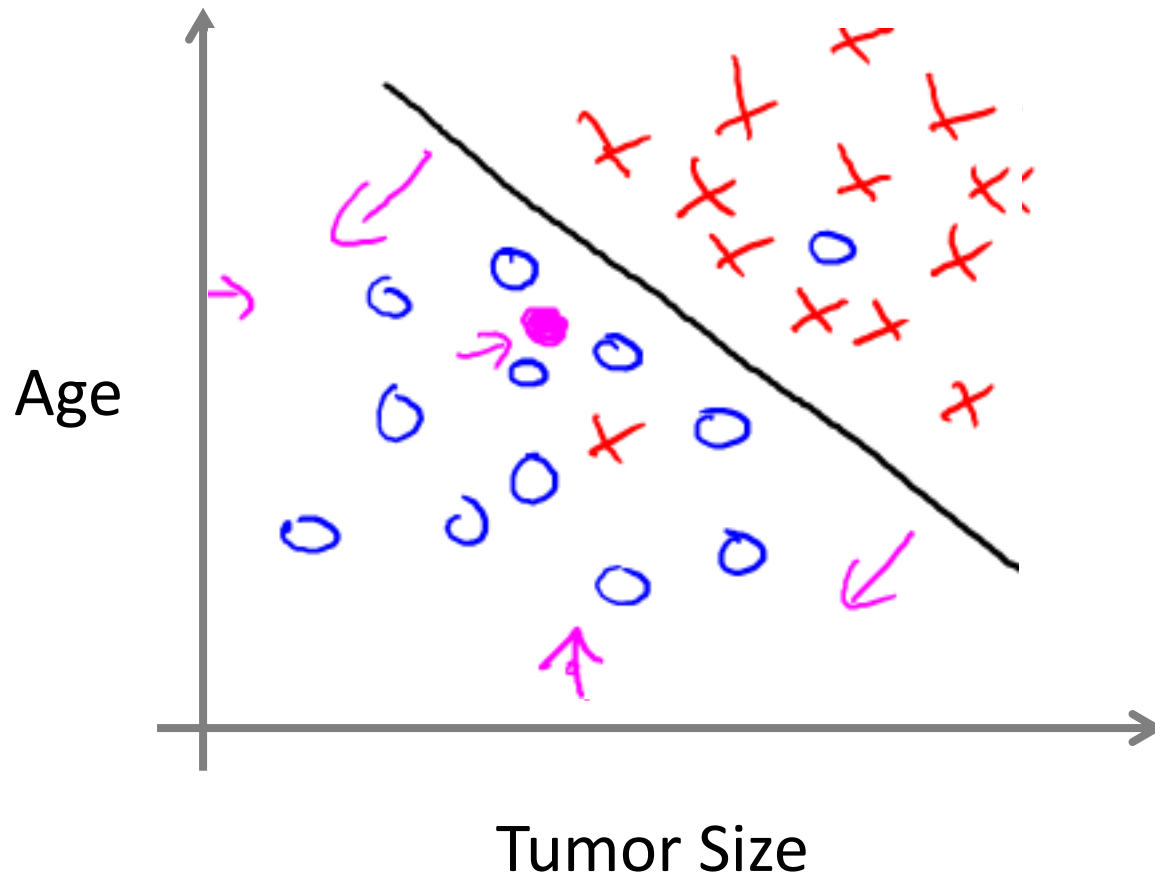
Regression: Predict continuous
valued output (price)

Cancer (malignant, benign)



Classification

Discrete valued
output (0 or 1)



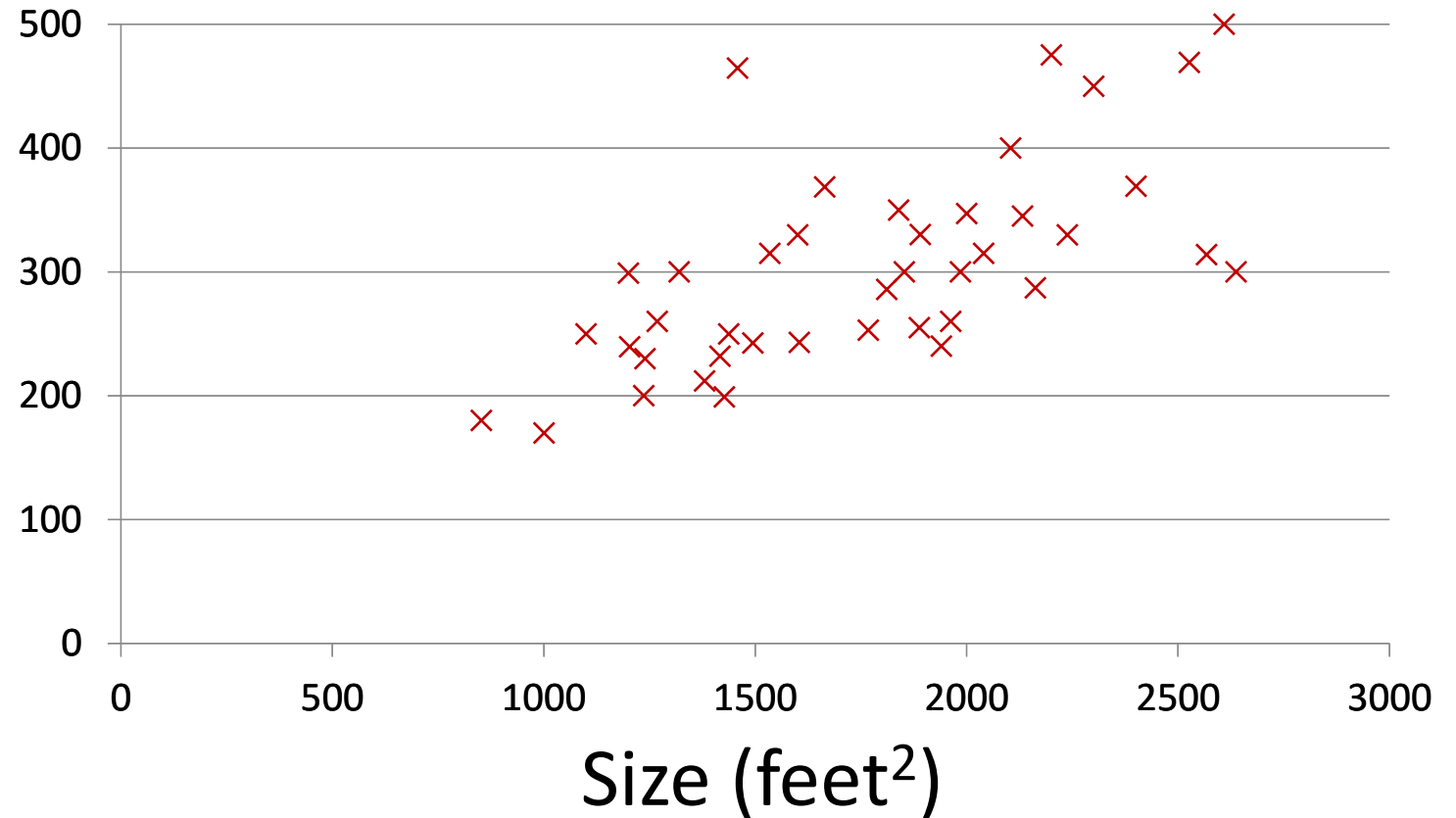
- Clump Thickness
- Uniformity of Cell Size
- Uniformity of Cell Shape
- ...

Linear regression
with one variable

Model representation

Housing Prices (Portland, OR)

Price
(in 1000s
of dollars)



Supervised Learning

Given the “right answer” for each example in the data.

Regression Problem

Predict real-valued output

Training set of housing prices	Size in feet ² (x)	Price (\$) in 1000's (y)
	2104	460
	1416	232
	1534	315
	852	178
	...	...

Notation:

m = Number of training examples

x's = “input” variable / features

y's = “output” variable / “target” variable

Training Set



Learning Algorithm



Size of
house



h



Estimated
price

How do we represent h ?

Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Linear regression with one variable.
Univariate linear regression.

Linear regression
with one variable

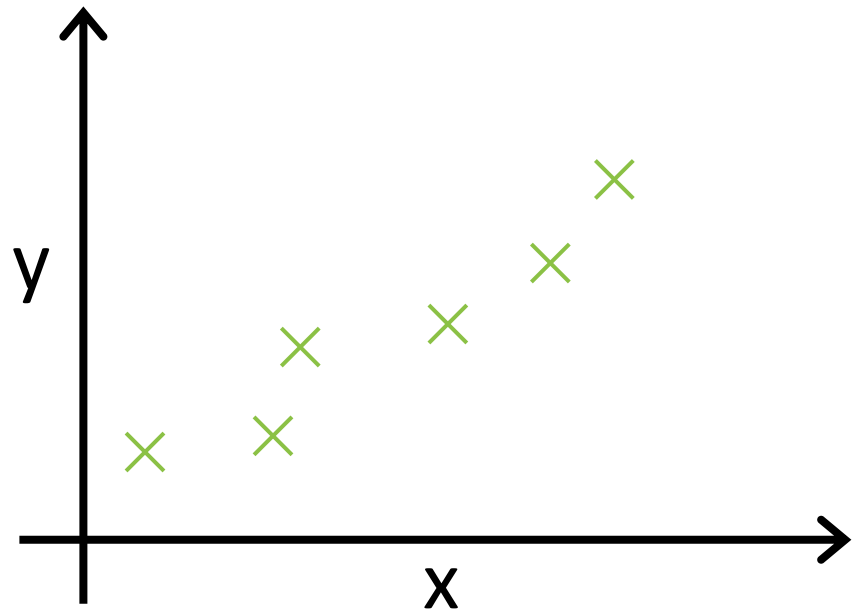
Cost function

Training Set	Size in feet ² (x)	Price (\$) in 1000's (y)
	2104	460
	1416	232
	1534	315
	852	178
	...	...

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

θ_i 's: Parameters

How to choose θ_i 's ?



Cost Function
(Squared Error function):

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Goal: minimize $J(\theta_0, \theta_1)$
 θ_0, θ_1

Idea: Choose θ_0, θ_1 so that
 $h_{\theta}(x)$ is close to y for
our training examples (x, y)

Linear regression
with one variable

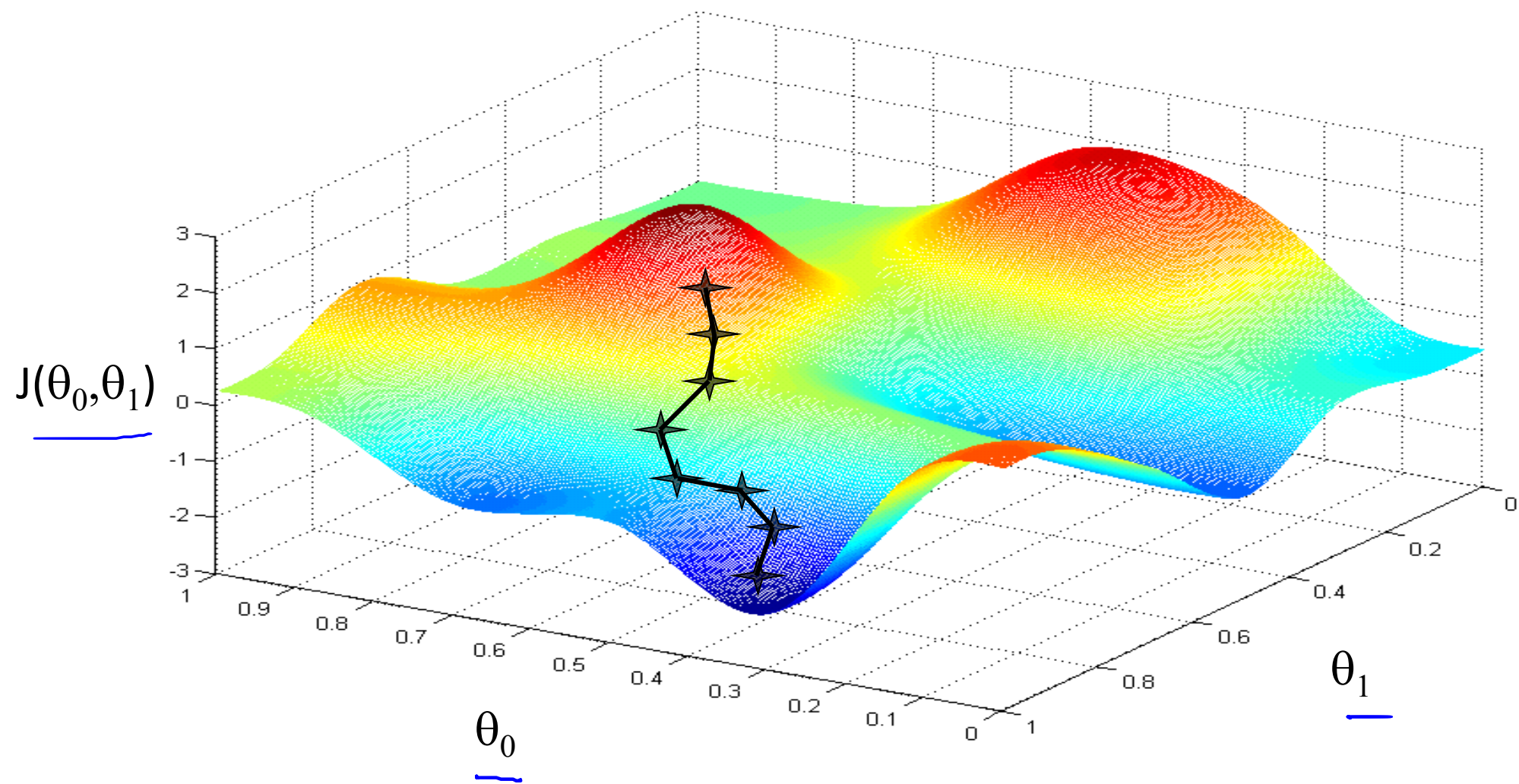
Gradient Descent

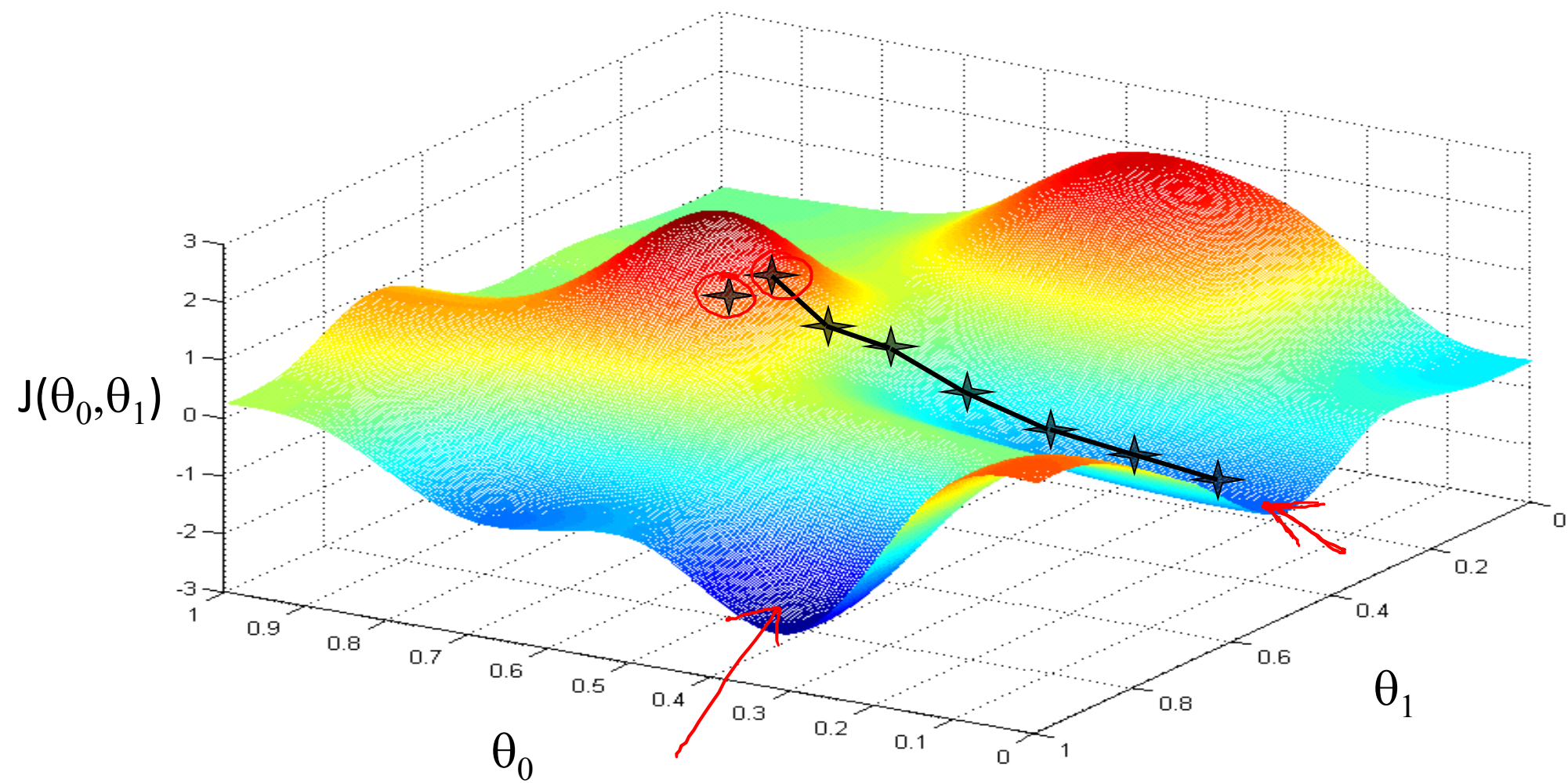
Have some function $J(\theta_0, \theta_1)$

Want $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

Outline:

- Start with some θ_0, θ_1
- Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$
until we hopefully end up at a minimum





Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ (for $j = 0$ and $j = 1$)
}

Correct: Simultaneous update

```
temp0 :=  $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$   
temp1 :=  $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$   
 $\theta_0 :=$  temp0  
 $\theta_1 :=$  temp1
```

Incorrect:

```
temp0 :=  $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$   
 $\theta_0 :=$  temp0  
temp1 :=  $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$   
 $\theta_1 :=$  temp1
```

Linear Regression with
multiple variables

Gradient descent for
multiple variables

Hypothesis: $h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \dots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \dots, \theta_n)$$

} (simultaneously update for every $j = 0, \dots, n$)

Gradient Descent

Previously (n=1):

Repeat {

$$\theta_0 := \theta_0 - \underbrace{\alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm ($n \geq 1$):

Repeat {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for
 $j = 0, \dots, n$)

}

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

...